

Unit NO.4 Functions

4.1 concept and need of function advantages with Examples

Concept of Function

- A **function** is a block of code written to perform a **specific task**.
- It helps in **dividing a large program into smaller parts**.
- A function runs only when it is **called** in the program.
- It improves **program structure and readability**.
- Functions help us **reuse the same code multiple times**.
- Each function has:
 - Function name
 - Return type
 - Body (set of statements)

Syntax of Function

```
return_type function_name()  
  
{  
  
    // statements  
  
}
```

Functions are needed in programming :

1. To **avoid repetition** of same code.
2. To **break large programs** into smaller manageable parts.
3. To make programs **easy to understand and write**.
4. To simplify **debugging and testing**.
5. To improve **program efficiency**.
6. To make modification or updating **easy**.
7. To save **time and effort** while programming.

Advantages of Using Functions

1. **Code Reusability** – Write once, use many times.
2. **Modularity** – Program divided into small sections.
3. **Easy Debugging** – Errors can be located easily.
4. **Improved Readability** – Program looks clean and structured.
5. **Easy Maintenance** – Changes required only in one place.
6. **Reduces Program Length** – No need to rewrite same logic.
7. **Better Logical Organization** – Each task handled separately.

Unit NO.4 Functions

Example:

```
#include <stdio.h>

void add()

{
    int a=5,b=10,sum;

    sum=a+b;

    printf("Sum = %d\n",sum);
}

void main()

{
    add(); // function call

    add(); // reused again
}
```

4.2 Library Functions:

- **Library functions** are the **predefined functions** already available in C.
- They are stored in **C standard libraries**.
- These functions help programmers perform common tasks **without writing code from scratch**.
- To use them, we must include the required **header file** using `#include`.

Examples of Library Functions

Function	Purpose	Header File
printf()	Display output	stdio.h
scanf()	Take input	stdio.h
strlen()	Find length of string	string.h
sqrt()	Find square root	math.h
clrscr()	Clear screen (Turbo C)	conio.h
getch()	Hold screen	conio.h

Unit NO.4 Functions

Example Program Using Library Functions

```
#include <stdio.h>
#include <conio.h>

void main()
{
    int a,b,sum;

    clrscr();

    printf("Enter two numbers: ");
    scanf("%d%d",&a,&b);

    sum=a+b;

    printf("Sum = %d",sum);

    getch();
}
```

Advantages of Library Functions

1. **Saves Time** – No need to write common code again.
2. **Reduces Program Length** – Ready-made functions simplify programs.
3. **Easy to Use** – Just include header file and call function.
4. **Reliable** – Already tested and error-free.
5. **Improves Productivity** – Faster program development.
6. **No Need to Remember Logic** – Function already defined.
7. **Makes Program Simple and Readable.**

4.2.1 Maths Functions

Math functions are **predefined functions** available in C to perform mathematical calculations like square root, power, rounding, etc.

These functions are declared in the header file:

```
#include <math.h>
```

Common Maths Functions in C

Advantages of Using Math Functions

1. No need to write formulas manually
2. Accurate and fast calculations
3. Saves programming time
4. Makes program short and simple

Unit NO.4 Functions

5. Useful in scientific and engineering programs

4.2.2 String Handling Functions

Definition

- `strlen()` is a **library function** used to find the **length of a string**.
- It counts the number of characters in a string **excluding the null character (\0)**.
- It is declared in the header file:

#include <string.h>

Syntax:

`strlen(string_name);`

Example Program to Find Length of String

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char str[50];
    int len;

    clrscr();

    printf("Enter a string: ");
    scanf("%s", str); // Input string (single word in Turbo C)

    len = strlen(str); // Find length using strlen()

    printf("Length of the string = %d", len);

    getch();
}
```

Output

Enter a string: Hello
Length of the string = 5

Advantages of `strlen()`

- Automatically calculates string length.
- Saves time (no need to count manually).

Unit NO.4 Functions

- Makes string programs simple and accurate.

strcpy() Function in C

Definition

- strcpy() is a **library function** used to **copy one string into another string**.
- It copies the **source string** to the **destination string** including the null character (\0).
- It is declared in the header file:

```
#include <string.h>
```

Syntax

```
strcpy(destination, source);
```

- source → Original string
- destination → String where content is copied

Example Program to Copy One String into Another

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char str1[50], str2[50];

    clrscr();

    printf("Enter first string: ");
    scanf("%s", str1); // Input string (single word)

    strcpy(str2, str1); // Copy str1 into str2

    printf("First String = %s\n", str1);
    printf("Second String (Copied) = %s\n", str2);

    getch();
}
```

Sample Output

```
Enter first string: Hello
First String = Hello
```

Unit NO.4 Functions

Second String (Copied) = Hello

Advantages of strcpy()

- Quickly copies strings without writing loops.
- Saves programming time.
- Simple and easy to use.
- Reduces chances of manual copying errors.

strcat() Function in C

Definition

- strcat() is a **library function** used to **concatenate (join) two strings**.
- It adds the **second string at the end of the first string**.
- It is declared in the header file:

```
#include <string.h>
```

Syntax

```
strcat(string1, string2);
```

- string1 → Destination string (where second string is added)
- string2 → Source string (which will be appended)

Example Program to Concatenate Two Strings

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char str1[50], str2[50];

    clrscr();

    printf("Enter first string: ");
    scanf("%s", str1); // Input first string

    printf("Enter second string: ");
```

Unit NO.4 Functions

```
scanf("%s", str2); // Input second string

strcat(str1, str2); // Join str2 to str1

printf("Concatenated String = %s", str1);

getch();
}
```

Sample Output

```
Enter first string: Hello
Enter second string: World
Concatenated String = HelloWorld
```

Advantages of strcat()

- Easily joins two strings.
- No need to write loops for concatenation.
- Saves time and simplifies string handling.

strcmp() Function in C

Definition

- strcmp() is a **library function** used to **compare two strings**.
- It checks whether the strings are **equal or different**.
- It is declared in the header file:

```
#include <string.h>
```

Syntax

```
strcmp(string1, string2);
```

Return Value of strcmp()

Return Value	Meaning
0	Both strings are equal
< 0	First string is smaller than second
> 0	First string is greater than second

Unit NO.4 Functions

Example Program to Compare Two Strings

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main()
{
    char str1[50], str2[50];
    int result;

    clrscr();

    printf("Enter first string: ");
    scanf("%s", str1);

    printf("Enter second string: ");
    scanf("%s", str2);

    result = strcmp(str1, str2); // Compare strings

    if(result == 0)
        printf("Strings are Equal");
    else
        printf("Strings are Not Equal");

    getch();
}
```

Sample Output 1

```
Enter first string: Hello
Enter second string: Hello
Strings are Equal
```

Sample Output 2

```
Enter first string: Hello
Enter second string: Hi
Strings are Not Equal
```

Advantages of strcmp()

- Quickly compares two strings.
- No need to write manual comparison loop.
- Useful in searching, sorting, and validation programs.

3.3 Miscellaneous Functions: getchar(), putchar(), malloc(), calloc()

Unit NO.4 Functions

getchar()

- Used to read a **single character** from the keyboard.
- Declared in stdio.h.

putchar()

- Used to **display a single character** on the screen.
- Declared in stdio.h.

malloc()

- Dynamically allocates **single block of memory** at runtime.
- Memory contains **garbage values** and is declared in stdlib.h.

calloc()

- Dynamically allocates memory for **multiple elements** at runtime.
- Initializes allocated memory to **zero** and is declared in stdlib.h.

User Defined Function

Function definition and prototype

Definition

- A **User Defined Function** is a function created by the programmer to perform a specific task.
- It helps to reuse code and makes the program easier to understand.

Parts of User Defined Function

1.Function Prototype (Declaration)

- It tells the compiler **what the function will do before it is actually defined** in the program.
- It helps the compiler check the **correct data type, number of arguments, and function name** during compilation.
- It is written **before main()** so the function can be called without errors.

Unit NO.4 Functions

Syntax:

```
return_type function_name(parameters);
```

Example:

```
void add(int, int);
```

2.Function Definition (Function Body)

- It contains the actual code (statements) that performs the task.

Syntax:

```
return_type function_name(parameters)
{
    // function body (statements)
}
```

Example:

```
void add(int a, int b)
{
    int sum;
    sum = a + b;
    printf("Sum = %d", sum);
}
```

3.Function Call

- It is used to execute the function in main().

Syntax:

```
function_name(arguments);
```

Example:

```
add(5, 10);
```

Program Example

```
#include <stdio.h>
#include <conio.h>
```

```
void add(int, int); // Function Prototype
```

```
void main()
```

Unit NO.4 Functions

```
{
    clrscr();

    add(5, 10);    // Function Call

    getch();
}

void add(int a, int b) // Function Definition
{
    int sum;
    sum = a + b;
    printf("Sum = %d", sum);
}
```

4.2.3 Calling a function

Calling a Function (Function Call)

Explanation

1. Function calling means **invoking or executing a function** from main() or another function.
2. When a function is called, the control of the program **transfers to that function**.
3. The function performs the given task and then **returns control back** to the calling function.
4. It helps in **reusing code** without writing the same logic again.

Program to Find Area of a Circle Using Function

```
#include <stdio.h>
#include <conio.h>

void area(); // Function Prototype

void main()
{
    clrscr();

    area(); // Function Call

    getch();
}

void area() // Function Definition
{
    float r, a;
```

Unit NO.4 Functions

```
printf("Enter radius of circle: ");
scanf("%f", &r);

a = 3.14 * r * r;

printf("Area of Circle = %.2f", a);
}
```

Output

Enter radius of circle: 5
Area of Circle = 78.50

Program: Factorial of n Numbers Using Function

```
#include <stdio.h>

#include <conio.h>

/* Function Declaration */

long int factorial(int);

/* Main Function */

void main()

{

    int i, num, n;

    long int result;

    clrscr();

    printf("How many numbers you want to find factorial? ");

    scanf("%d", &n);
```

Unit NO.4 Functions

```
for(i = 1; i <= n; i++)  
  
{  
  
    printf("\nEnter number %d: ", i);  
  
    scanf("%d", &num);  
  
  
    result = factorial(num); /* Function Call */  
  
    printf("Factorial of %d = %ld\n", num, result);  
  
}  
  
getch();  
  
}  
  
/* Function Definition */  
  
long int factorial(int num)  
  
{  
  
    int i;  
  
    long int fact = 1;  
  
  
    for(i = 1; i <= num; i++)  
  
    {  
  
        fact = fact * i;  
  
    }  
  
}
```

Unit NO.4 Functions

```
    return fact;  
}
```

Program: Sum of Natural Numbers

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int n, i;
```

```
    long int sum = 0;
```

```
    clrscr();
```

```
    printf("Enter a natural number: ");
```

```
    scanf("%d", &n);
```

```
    for(i = 1; i <= n; i++)
```

```
    {
```

```
        sum = sum + i;
```

```
    }
```

```
    printf("Sum of first %d natural numbers = %ld", n, sum);
```

```
    getch();
```

```
}
```

Unit NO.4 Functions

Program: To find Fibonacci Series of number

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
    int n, i;
```

```
    long int a = 0, b = 1, c;
```

```
    clrscr();
```

```
    printf("Enter number of terms: ");
```

```
    scanf("%d", &n);
```

```
    printf("Fibonacci Series: ");
```

```
    for(i = 1; i <= n; i++)
```

```
    {
```

```
        printf("%ld ", a);
```

```
        c = a + b;
```

```
        a = b;
```

```
        b = c;
```

```
    }
```

```
    getch();
```

```
}
```

Unit NO.4 Functions

4.3.3 Passing Arguments to function

- Arguments are values passed to a function
- Arguments are also called parameters
- Any number of arguments can be passed
- Type, number and order of arguments must match
- Arguments are written inside parentheses

Example:

```
void large(int a)
```

Example 4.23 – Program to Find Area of Circle Using Function

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void Area(float radius)
```

```
{
```

```
    float area, pi;
```

```
    pi = 3.14;
```

```
    area = pi * radius * radius;
```

```
    printf("\nThe area of circle is %f", area);
```

```
}
```

```
void main()
```

```
{
```

```
    float r;
```

```
    clrscr();
```

```
    printf("Enter radius of circle: ");
```


Unit NO.4 Functions

```
scanf("%f", &r);

Area(r);


printf("\nEnter radius of circle: ");

scanf("%f", &r);

Area(r);


getch();

}
```

Example 4.30 – Program to Find Sum of Natural Numbers

```
#include <stdio.h>

#include <conio.h>


void sum(int n)

{

    int i, total=0;


    for(i=1; i<=n; i++)

        total = total + i;


    printf("\nThe sum is = %d", total);

}


void main()
```

Unit NO.4 Functions

```
{  
  
    int k;  
  
    clrscr();  
  
  
    printf("Enter the number: ");  
  
    scanf("%d", &k);  
  
    sum(k);  
  
  
  
    printf("\nEnter the number: ");  
  
    scanf("%d", &k);  
  
    sum(k);  
  
  
  
    getch();  
}
```

Example 4.31 – Fibonacci Series Using Function

```
#include <stdio.h>  
  
#include <conio.h>  
  
  
  
void fibonacci(int n)  
{  
  
    int f1=1, f2=1, f3, i;  
  
  
  
    printf("%d %d ", f1, f2);  
  
  
  
  
    for(i=3; i<=n; i++)
```

Unit NO.4 Functions

```
{  
    f3 = f1 + f2;  
    printf("%d ", f3);  
    f1 = f2;  
    f2 = f3;  
}  
}
```

```
void main()  
{  
    int n;  
    clrscr();  
  
    printf("Enter number of terms: ");  
    scanf("%d", &n);  
  
    fibonacci(n);  
  
    getch();  
}
```

3.4.4 Returning the Values from Function

- A function can return a value to the calling function.
- The return statement is used to send value back.
- The return type must be declared before function name.
- Only one value can be returned at a time.

Syntax:

return_type function_name(parameters)

Unit NO.4 Functions

```
{  
    return value;  
}
```

Example:

```
int sum(int a, int b)  
{  
    return a+b;  
}
```

Example 1: Program to Add Two Numbers Using Return Value

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int add(int a, int b)
```

```
{  
  
    int c;  
  
    c = a + b;  
  
    return c;  
}
```

```
void main()
```

```
{  
  
    int x, y, result;  
  
    clrscr();  
  
    printf("Enter two numbers: ");  
  
    scanf("%d %d", &x, &y);  
  
    result = add(x,y);
```

Unit NO.4 Functions

```
printf("Sum is = %d", result);
```

```
getch();
```

```
}
```

Program: Calculate Grand Area of Two Circles

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
float area(float r)
```

```
{
```

```
    float a;
```

```
    a = 3.14 * r * r;
```

```
    return a;
```

```
}
```

```
void main()
```

```
{
```

```
    float r1, r2, area1, area2, grand_total;
```

```
    clrscr();
```

```
    printf("Enter radius of first circle: ");
```

```
    scanf("%f", &r1);
```

```
    printf("Enter radius of second circle: ");
```

Unit NO.4 Functions

```
scanf("%f", &r2);
```

```
area1 = area(r1);
```

```
area2 = area(r2);
```

```
grand_total = area1 + area2;
```

```
printf("\nArea of first circle = %f", area1);
```

```
printf("\nArea of second circle = %f", area2);
```

```
printf("\nGrand Total Area = %f", grand_total);
```

```
getch();
```

```
}
```

Program: Find Largest of Two Numbers

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int largest(int a, int b)
```

```
{
```

```
    if(a > b)
```

```
        return a;
```

```
    else
```

```
        return b;
```

```
}
```

Unit NO.4 Functions

```
void main()

{

    int x, y, result;

    clrscr();


    printf("Enter two numbers: ");

    scanf("%d %d", &x, &y);


    result = largest(x, y);


    printf("Largest number is = %d", result);


    getch();

}
```

4.3.5 Types of function

Function with Arguments (No Return Value)

Definition:

A **function with arguments** receives values from the calling function but does not return any value.

- 1 It takes parameters inside brackets.
- 2 Values are passed during function call.
- 3 It performs operation using given values.
- 4 It does not return any value (void type).

Example:

```
#include <stdio.h>
#include <conio.h>


void add(int a, int b) // Function with arguments
{
    int sum;
```

Unit NO.4 Functions

```
    sum = a + b;
    printf("Sum = %d", sum);
}

void main()
{
    clrscr();
    add(10, 20); // Passing arguments
    getch();
}
```

Function with No Arguments but Return Value

Definition:

A **function with no arguments but return value** does not take any input from the calling function, but it **returns a value** after performing the operation.

1. It does not take any parameters inside brackets.
2. It takes input inside the function body (if needed).
3. It returns a value using `return` statement.
4. The returned value is received in the calling function.

Example:

```
#include <stdio.h>
#include <conio.h>

int add() // No arguments but return value
{
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    sum = a + b;
    return sum; // Returning value
}

void main()
{
    int result;
    clrscr();

    result = add(); // Function call

    printf("Sum = %d", result);
    getch();
}
```


Unit NO.4 Functions

4.4 Parameter Passing to function

Parameter passing is the process of **transferring values or variables to a function at the time of calling it** so the function can use them.

Definition:

Call by Value is a method of parameter passing in which **a copy of the actual argument is passed to the function.**

In this method:

- Changes made inside the function **do not affect** the original variable.
- The function works on a **duplicate copy** of the data.

The value of the variable is copied into the formal parameter.

1. The function works on the copied value.
2. Original variables remain unchanged.
3. It provides data safety.
4. It is the default method in C language.

1.Illustration of Call by Value

```
#include <stdio.h>
#include <conio.h>

void change(int x) // x is formal parameter
{
    x = x + 10;
    printf("Value of x inside function = %d\n", x);
}

void main()
{
    int a = 20;
    clrscr();

    change(a); // a is actual parameter

    printf("Value of a in main function = %d", a);

    getch();
}
```

Unit NO.4 Functions

Output:

Value of x inside function = 30
Value of a in main function = 20

4.4.2 Passing pointers to a function (Call by reference)

Definition:

Call by Reference is a method of passing parameters where the **address of the variable is passed to the function** using pointers.

The function works on the original variable through its address.

1. The **address of the variable** is passed to the function using pointers.
2. Any change made inside the function **affects the original variable**.

Example

```
#include <stdio.h>
#include <conio.h>

void change(int *x, int *y)
{
    *x = *x + 10;
    *y = *y + 20;
}

void main()
{
    int a = 5, b = 10;
    clrscr();

    change(&a, &b);

    printf("Value of a = %d\n", a);
    printf("Value of b = %d", b);

    getch();
}
```

Output:

Value of a = 15
Value of b = 30

Here:

- &a and &b → addresses are passed

Unit NO.4 Functions

- *x and *y modify original variables
- So values of a and b change permanently

Difference Between Call by Value and Call by Reference

Call by Value	Call by Reference
1. Value of actual argument is passed.	1. Address of actual argument is passed.
2. Function works on copy of data.	2. Function works on original data using address.
3. Changes do not affect original variable.	3. Changes affect original variable.
4. Example calling: change(x, y);	4. Example calling: change(&x, &y);

4.47. Program to Find Whether Entered Number is Armstrong or Not

Definition:

An **Armstrong number** is a number in which **sum of the cubes of its digits is equal to the number itself**.

Example:

$$153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$$

C Program (Using Function)

```
#include <stdio.h>
#include <conio.h>

void armstrong(int num)
{
    int temp, rem, sum = 0;
    temp = num;

    while(temp != 0)
    {
        rem = temp % 10;
        sum = sum + (rem * rem * rem);
        temp = temp / 10;
    }
}
```

Unit NO.4 Functions

```
    if(sum == num)
        printf("Armstrong Number");
    else
        printf("Not an Armstrong Number");
}

void main()
{
    int number;
    clrscr();

    printf("Enter a number: ");
    scanf("%d", &number);

    armstrong(number);

    getch();
}
```

Output:

Enter a number: 153
Armstrong Number

4.5 Recursion

1. **Recursion** is a process in which a function **calls itself** again and again.
2. It continues calling itself **until a specific condition is satisfied**.
3. It is mainly used for **repetitive calculations**.
4. In recursion, each step is written in terms of the **previous result**.
5. Many repetitive problems can be solved using recursion.
6. To solve a problem using recursion, **two conditions are necessary**:
 - Recursive case
 - Base (stopping) condition
7. The **base condition** is very important, otherwise the function will run forever.
8. Example:
Factorial can be written as
$$n! = n \times (n - 1)!$$
and stopping condition is
$$1! = 1$$

Example of Recursive Function

```
void area()
{
    area(); // function calling itself
}
```

Unit NO.4 Functions

1.Program to Calculate Factorial Using Recursive Function

Definition:

Factorial of a number **n** is:

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

Recursive formula:

$$n! = n \times (n - 1)!$$

Base condition:

$$1! = 1$$

C Program (Recursive Method)

```
#include <stdio.h>
#include <conio.h>

long int fact(int n)
{
    if(n == 0 || n == 1) // Base condition
        return 1;
    else
        return n * fact(n - 1); // Recursive call
}

void main()
{
    int num;
    long int result;

    clrscr();

    printf("Enter a number: ");
    scanf("%d", &num);

    result = fact(num);

    printf("Factorial of %d = %ld", num, result);

    getch();
}
```

Unit NO.4 Functions

Output:

Enter a number: 5
Factorial of 5 = 120

4.6 Command line Arguments

Definition:

Command line arguments are the values passed to a program at the time of execution through the command prompt.

They are received by the `main()` function.

1. Command line arguments are passed while running the program from DOS/Command Prompt.
2. They are received using parameters in main function:
`int main(int argc, char *argv[])`
3. `argc` stores the number of arguments.
4. `argv` is an array of strings that stores the arguments.

4.6. Display Command Line Arguments

```
#include <stdio.h>
#include <conio.h>

void main(int argc, char *argv[])
{
    int i;
    clrscr();

    printf("Number of arguments = %d\n", argc);

    for(i = 0; i < argc; i++)
    {
        printf("Argument %d = %s\n", i, argv[i]);
    }

    getch();
}
```

Output:

Number of arguments = 3
Argument 0 = program
Argument 1 = 10
Argument 2 = 20

Unit NO.4 Functions

4.7 Scope and Lifetime of variables

Scope – It defines where a variable can be accessed in the program (inside a function or throughout the program).

Lifetime – It defines how long a variable stays in memory during the execution of the program.

Local Variable

Definition:

A **local variable** is declared inside a function or block and can be used **only inside that function**.

Example:

```
#include <stdio.h>
#include <conio.h>

void display()
{
    int a = 10; // local variable
    printf("Value of a = %d", a);
}

void main()
{
    clrscr();
    display();
    getch();
}
```

Explanation:

- Variable a is declared inside display().
- It cannot be used outside that function.
- Its lifetime is only while the function is running.

Global Variable

Definition: A global variable is declared outside all functions and can be used by all functions **in the program**.

Unit NO.4 Functions

Example:

```
#include <stdio.h>
#include <conio.h>

int a = 20; // global variable

void show()
{
    printf("Value of a = %d\n", a);
}

void main()
{
    clrscr();
    show();
    printf("Value of a in main = %d", a);
    getch();
}
```

4.8 Storage Classes

- a. **Automatic** identified by keyword auto
- b. **External** identified by keyword extern
- c. **Static** identified by keyword static
- d. **Register** identified by keyword register

Syntax:

storage_class data_type variable_name;

Examples:

auto int radius;

static int area;

extern char star;

register int count;

Unit NO.4 Functions

4.8.1 Automatic variables (Local Variables)

- 1 Automatic variables are declared inside a function or block.
- 2 They are identified by the keyword `auto` (it is optional because local variables are automatic by default).
- 3 Their scope is limited only to the function in which they are declared.
- 4 Their lifetime exists only during the execution of that function.

Example:

```
#include <stdio.h>
#include <conio.h>

void display()
{
    auto int a = 10; // automatic (local) variable
    printf("Value of a = %d", a);
}

void main()
{
    clrscr();
    display();
    getch();
}
```

Explanation:

- Variable `a` is declared inside `display()`.
- It cannot be used outside that function.
- It is destroyed after the function finishes execution.

4.8.2 External (Global) Variable

- 1 External variable is also called a **global variable**.
- 2 It is declared outside all functions.
- 3 It can be accessed by all functions in the program.
- 4 It is identified by the keyword `extern` (used to refer the global variable).
- 5 Its scope is throughout the program file.
- 6 Its lifetime is entire program execution.

Unit NO.4 Functions

Example:

```
#include <stdio.h>
#include <conio.h>

int a = 50; // Global variable

void display()
{
    extern int a; // referring global variable
    printf("Value of a in display = %d\n", a);
}

void main()
{
    clrscr();
    display();
    printf("Value of a in main = %d", a);
    getch();
}
```

Output:

Value of a in display = 50
Value of a in main = 50

Explanation:

- Variable a is declared outside all functions.
- It is accessed inside both display() and main().
- It exists until the program ends.

Program to Exchange (Swap) Two Variables Using Call by Reference

```
#include <stdio.h>
#include <conio.h>

void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}

void main()
{
    int a, b;
    clrscr();

    printf("Enter first number: ");
    scanf("%d", &a);
```

Unit NO.4 Functions

```
printf("Enter second number: ");  
scanf("%d", &b);  
  
swap(&a, &b); // Call by Reference  
  
printf("After swapping:\n");  
printf("First number = %d\n", a);  
printf("Second number = %d", b);  
  
getch();  
}
```

Sample Output:

```
Enter first number: 10  
Enter second number: 20  
After swapping:  
First number = 20  
Second number = 10
```

Explanation:

- Addresses of a and b are passed using &.
- Function uses pointers *x and *y.
- Original values are changed because address is passed.
- This is called **Call by Reference**.